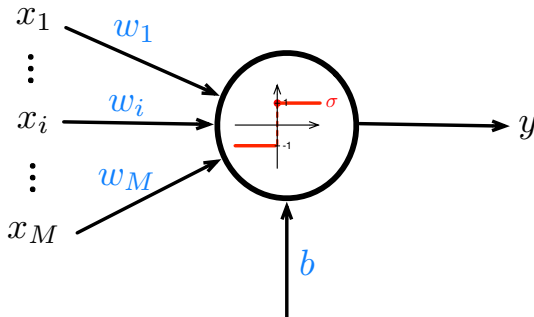


LE PERCEPTRON

Jérémie Cabessa
Laboratoire DAVID, UVSQ

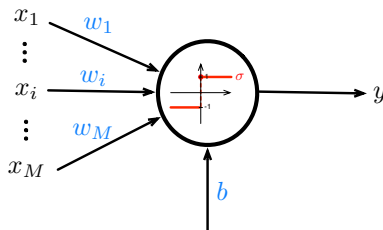
PERCEPTRON

- Le **perceptron** [Rosenblatt, 1957, Rosenblatt, 1958] est un simple neurone qui agit comme un *classifieur binaire*.



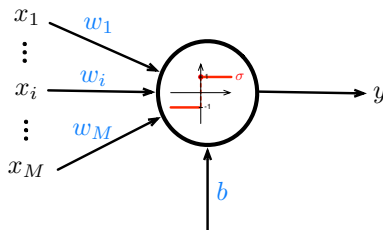
PERCEPTRON

- ▶ $\mathbf{x} = (x_1, \dots, x_M) \in \mathbb{R}^M$ sont les *inputs*.
- ▶ $\mathbf{w} = (w_1, \dots, w_M) \in \mathbb{R}^M$ et $b \in \mathbb{R}$ sont les *paramètres*: *poids synaptiques* et *biais*, respectivement.
- ▶ $y \in \{-1, +1\}$ est l'*output* (binaire).
- ▶ σ est la *fonction d'activation*.



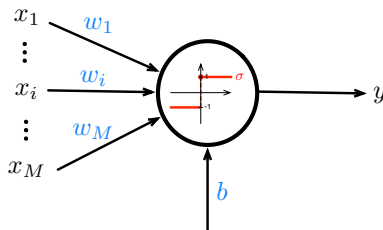
PERCEPTRON

- ▶ $\mathbf{x} = (x_1, \dots, x_M) \in \mathbb{R}^M$ sont les *inputs*.
- ▶ $\mathbf{w} = (w_1, \dots, w_M) \in \mathbb{R}^M$ et $b \in \mathbb{R}$ sont les *paramètres*: *poids synaptiques* et *biais*, respectivement.
- ▶ $y \in \{-1, +1\}$ est l'*output* (binaire).
- ▶ σ est la *fonction d'activation*.



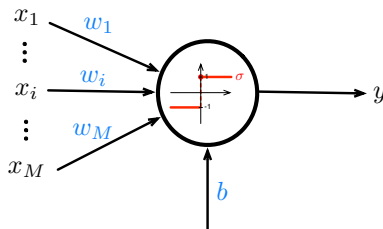
PERCEPTRON

- ▶ $\mathbf{x} = (x_1, \dots, x_M) \in \mathbb{R}^M$ sont les *inputs*.
- ▶ $\mathbf{w} = (w_1, \dots, w_M) \in \mathbb{R}^M$ et $b \in \mathbb{R}$ sont les *paramètres*: *poids synaptiques* et *biais*, respectivement.
- ▶ $y \in \{-1, +1\}$ est l'*output* (binaire).
- ▶ σ est la *fonction d'activation*.



PERCEPTRON

- ▶ $\mathbf{x} = (x_1, \dots, x_M) \in \mathbb{R}^M$ sont les *inputs*.
- ▶ $\mathbf{w} = (w_1, \dots, w_M) \in \mathbb{R}^M$ et $b \in \mathbb{R}$ sont les *paramètres*: *poids synaptiques* et *biais*, respectivement.
- ▶ $y \in \{-1, +1\}$ est l'*output* (binaire).
- ▶ σ est la *fonction d'activation*.

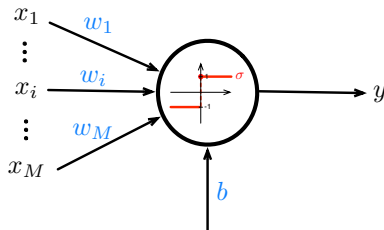


PERCEPTRON

- La dynamique du perceptron est la suivante:

$$y = \sigma \left(\sum_{i=1}^M w_i x_i + b \right) = \sigma (\mathbf{w}^T \mathbf{x} + b) = \begin{cases} +1, & \text{if } \mathbf{w}^T \mathbf{x} + b \geq 0 \\ -1, & \text{if } \mathbf{w}^T \mathbf{x} + b < 0. \end{cases}$$

où $\mathbf{x} = (x_1, \dots, x_M)$ et $\mathbf{w} = (w_1, \dots, w_M)$.



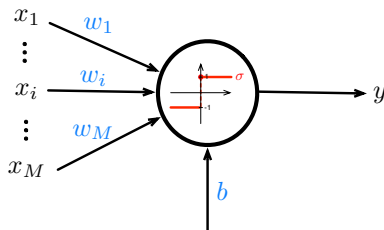
PERCEPTRON

- Par abus de langage, posons:

$$\mathbf{x} := (1, x_1, \dots, x_M) \text{ et } \mathbf{w} := (b, w_1, \dots, w_M).$$

- La dynamique du perceptron s'écrit alors:

$$y = \sigma(\mathbf{w}^T \mathbf{x}) = \begin{cases} +1, & \text{if } \mathbf{w}^T \mathbf{x} \geq 0 \\ -1, & \text{if } \mathbf{w}^T \mathbf{x} < 0. \end{cases}$$



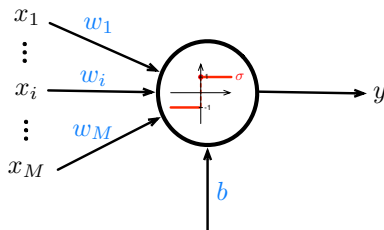
PERCEPTRON

- ▶ Par abus de langage, posons:

$$\mathbf{x} := (1, x_1, \dots, x_M) \text{ et } \mathbf{w} := (b, w_1, \dots, w_M).$$

- ▶ La dynamique du perceptron s'écrit alors:

$$y = \sigma(\mathbf{w}^T \mathbf{x}) = \begin{cases} +1, & \text{if } \mathbf{w}^T \mathbf{x} \geq 0 \\ -1, & \text{if } \mathbf{w}^T \mathbf{x} < 0. \end{cases}$$



REMARQUE

- Soient $\mathbf{w} = (b, w_1, \dots, w_M) \in \mathbb{R}^{M+1}$ les paramètres du perceptron.
- Le vecteur $\mathbf{w}$ définit un *hyperplan* dans l'espace des inputs $\mathbb{R}^M$ (en non $\mathbb{R}^{M+1}$) dont l'équation est

$$\mathbf{w}^T \mathbf{x} = 0 \quad \text{i.e.,} \quad \sum_{i=1}^M w_i x_i + b = 0.$$

- $\mathbf{w}' := (w_1, \dots, w_M)$ est le vecteur normal de cet hyperplan.
- Par définition, le perceptron de paramètres $\mathbf{w}$ classe les points de $\mathbb{R}^M$ de part et d'autre de cet hyperplan.

REMARQUE

- ▶ Soient $\mathbf{w} = (b, w_1, \dots, w_M) \in \mathbb{R}^{M+1}$ les paramètres du perceptron.
- ▶ Le vecteur $\mathbf{w}$ définit un *hyperplan* dans l'espace des inputs $\mathbb{R}^M$ (en non $\mathbb{R}^{M+1}$) dont l'équation est

$$\mathbf{w}^T \mathbf{x} = 0 \quad \text{i.e.,} \quad \sum_{i=1}^M w_i x_i + b = 0.$$

- ▶ $\mathbf{w}' := (w_1, \dots, w_M)$ est le vecteur normal de cet hyperplan.
- ▶ Par définition, le perceptron de paramètres $\mathbf{w}$ classe les points de $\mathbb{R}^M$ de part et d'autre de cet hyperplan.

REMARQUE

- ▶ Soient $\mathbf{w} = (b, w_1, \dots, w_M) \in \mathbb{R}^{M+1}$ les paramètres du perceptron.
- ▶ Le vecteur $\mathbf{w}$ définit un *hyperplan* dans l'espace des inputs $\mathbb{R}^M$ (en non $\mathbb{R}^{M+1}$) dont l'équation est

$$\mathbf{w}^T \mathbf{x} = 0 \quad \text{i.e.,} \quad \sum_{i=1}^M w_i x_i + b = 0.$$

- ▶ $\mathbf{w}' := (w_1, \dots, w_M)$ est le vecteur normal de cet hyperplan.
- ▶ Par définition, le perceptron de paramètres $\mathbf{w}$ classe les points de $\mathbb{R}^M$ de part et d'autre de cet hyperplan.

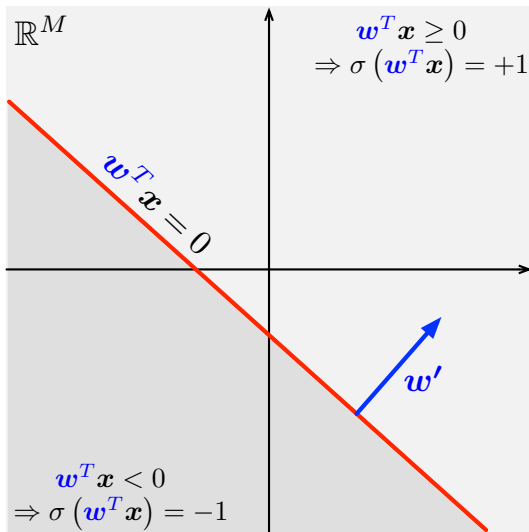
REMARQUE

- ▶ Soient $\mathbf{w} = (b, w_1, \dots, w_M) \in \mathbb{R}^{M+1}$ les paramètres du perceptron.
- ▶ Le vecteur $\mathbf{w}$ définit un *hyperplan* dans l'espace des inputs $\mathbb{R}^M$ (en non $\mathbb{R}^{M+1}$) dont l'équation est

$$\mathbf{w}^T \mathbf{x} = 0 \quad \text{i.e.,} \quad \sum_{i=1}^M w_i x_i + b = 0.$$

- ▶ $\mathbf{w}' := (w_1, \dots, w_M)$ est le vecteur normal de cet hyperplan.
- ▶ Par définition, le perceptron de paramètres $\mathbf{w}$ classe les points de $\mathbb{R}^M$ de part et d'autre de cet hyperplan.

REMARQUE



ENTRAÎNEMENT (TRAINING)

- Soit un train set

$$\mathcal{S} = \{(\mathbf{x}_k, y_k) \in \mathbb{R}^M \times \{-1, +1\} : k = 1, \dots, K\}.$$

- L'*entraînement* d'un perceptron (*training*): déterminer des poids $\hat{\mathbf{w}}$ tels que les points du train set soient bien classifiés, i.e.:

$$\text{Si } y_k = +1, \text{ alors } \sigma(\hat{\mathbf{w}}^T \mathbf{x}_k) = +1$$

$$\text{Si } y_k = -1, \text{ alors } \sigma(\hat{\mathbf{w}}^T \mathbf{x}_k) = -1$$

- Si les points ne sont pas linéairement séparables, on peut fixer un critère d'arrêt...

ENTRAÎNEMENT (TRAINING)

- Soit un train set

$$\mathcal{S} = \{(\mathbf{x}_k, y_k) \in \mathbb{R}^M \times \{-1, +1\} : k = 1, \dots, K\}.$$

- L'*entraînement* d'un perceptron (*training*): déterminer des poids $\hat{\mathbf{w}}$ tels que les points du train set soient bien classifiés, i.e.:

$$\text{Si } y_k = +1, \text{ alors } \sigma(\hat{\mathbf{w}}^T \mathbf{x}_k) = +1$$

$$\text{Si } y_k = -1, \text{ alors } \sigma(\hat{\mathbf{w}}^T \mathbf{x}_k) = -1$$

- Si les points ne sont pas linéairement séparables, on peut fixer un critère d'arrêt...

ENTRAÎNEMENT (TRAINING)

- Soit un train set

$$\mathcal{S} = \{(\mathbf{x}_k, y_k) \in \mathbb{R}^M \times \{-1, +1\} : k = 1, \dots, K\}.$$

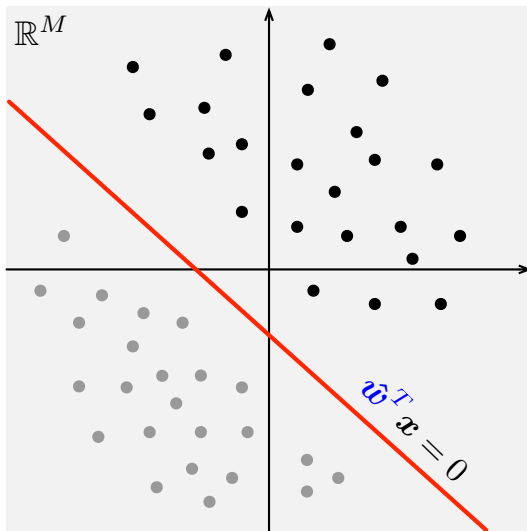
- L'*entraînement* d'un perceptron (*training*): déterminer des poids $\hat{\mathbf{w}}$ tels que les points du train set soient bien classifiés, i.e.:

$$\text{Si } y_k = +1, \text{ alors } \sigma(\hat{\mathbf{w}}^T \mathbf{x}_k) = +1$$

$$\text{Si } y_k = -1, \text{ alors } \sigma(\hat{\mathbf{w}}^T \mathbf{x}_k) = -1$$

- Si les points ne sont pas linéairement séparables, on peut fixer un critère d'arrêt...

ENTRAÎNEMENT (TRAINING)



ENTRAÎNEMENT (TRAINING)

Algorithm 1: Training algorithm of the perceptron

Data: dataset = $\{(\mathbf{x}_i, y_i) : i = 1, \dots, K\}$

$w := (0, \dots, 0, 0) = \mathbf{0}$

for $e = 1$ to nb_epochs do

 for $k = 1$ to K do

 if $y_k = -1$ and $\sigma(w^T x_k) = +1$ then

$w := w - x_k$

 else if $y_k = +1$ and $\sigma(w^T x_k) = -1$ then

$w := w + x_k$

 end

 end

end

return w

ENTRAÎNEMENT (TRAINING)

Algorithm 1: Training algorithm of the perceptron

Data: dataset = $\{(x_i, y_i) : i = 1, \dots, K\}$

$w := (0, \dots, 0, 0) = \mathbf{0}$

for $e = 1$ to nb_epochs do

 for $k = 1$ to K do

 if $y_k = -1$ and $\sigma(w^T x_k) = +1$ then

$w := w - x_k$

 else if $y_k = +1$ and $\sigma(w^T x_k) = -1$ then

$w := w + x_k$

 end

 end

end

return w

ENTRAÎNEMENT (TRAINING)

Algorithm 1: Training algorithm of the perceptron

Data: dataset = $\{(x_i, y_i) : i = 1, \dots, K\}$

$w := (0, \dots, 0, 0) = \mathbf{0}$

for $e = 1$ *to* nb_epochs **do**

for $k = 1$ *to* K **do**

if $y_k = -1$ *and* $\sigma(w^T x_k) = +1$ **then**

$w := w - x_k$

else if $y_k = +1$ *and* $\sigma(w^T x_k) = -1$ **then**

$w := w + x_k$

end

end

end

return w

ENTRAÎNEMENT (TRAINING)

Algorithm 1: Training algorithm of the perceptron

Data: dataset = $\{(x_i, y_i) : i = 1, \dots, K\}$

$w := (0, \dots, 0, 0) = \mathbf{0}$

for $e = 1$ *to* nb_epochs **do**

for $k = 1$ *to* K **do**

if $y_k = -1$ *and* $\sigma(w^T x_k) = +1$ **then**

$w := w - x_k$

else if $y_k = +1$ *and* $\sigma(w^T x_k) = -1$ **then**

$w := w + x_k$

end

end

end

return w

ENTRAÎNEMENT (TRAINING)

Algorithm 1: Training algorithm of the perceptron

Data: dataset = $\{(x_i, y_i) : i = 1, \dots, K\}$

$w := (0, \dots, 0, 0) = \mathbf{0}$

for $e = 1$ *to* nb_epochs **do**

for $k = 1$ *to* K **do**

if $y_k = -1$ *and* $\sigma(w^T x_k) = +1$ **then**

$w := w - x_k$

else if $y_k = +1$ *and* $\sigma(w^T x_k) = -1$ **then**

$w := w + x_k$

end

end

end

return w

ENTRAÎNEMENT (TRAINING)

Algorithm 1: Training algorithm of the perceptron

Data: dataset = $\{(x_i, y_i) : i = 1, \dots, K\}$ $w := (0, \dots, 0, 0) = \mathbf{0}$ **for** $e = 1$ *to* nb_epochs **do** **for** $k = 1$ *to* K **do** **if** $y_k = -1$ *and* $\sigma(w^T x_k) = +1$ **then** $w := w - x_k$ **else if** $y_k = +1$ *and* $\sigma(w^T x_k) = -1$ **then** $w := w + x_k$ **end** **end****end****return** w

ENTRAÎNEMENT (TRAINING)

Algorithm 2: Training algorithm of the perceptron (rewriting)

Data: dataset = $\{(\mathbf{x}_i, y_i) : i = 1, \dots, K\}$

```
 $w := (0, \dots, 0, 0) = \mathbf{0}$ 
for  $e = 1$  to  $nb\_epochs$  do
    for  $k = 1$  to  $K$  do
        if  $y_k \cdot \sigma(w^T x_k) < 0$  then
             $w := w + y_k \cdot x_k$ 
        end
    end
end
return  $w$ 
```

- On peut utiliser un *learning rate* $\gamma > 0$, ce qui modifie la règle de mise à jour en $w := w + \gamma \cdot y_k \cdot x_k$

ENTRAÎNEMENT (TRAINING)

Algorithm 2: Training algorithm of the perceptron (rewriting)

Data: dataset = $\{(\mathbf{x}_i, y_i) : i = 1, \dots, K\}$

$\mathbf{w} := (0, \dots, 0, 0) = \mathbf{0}$

```
for  $e = 1$  to  $nb\_epochs$  do
  for  $k = 1$  to  $K$  do
    if  $y_k \cdot \sigma(\mathbf{w}^T \mathbf{x}_k) < 0$  then
       $\mathbf{w} := \mathbf{w} + y_k \cdot \mathbf{x}_k$ 
    end
  end
end
return  $\mathbf{w}$ 
```

- On peut utiliser un *learning rate* $\gamma > 0$, ce qui modifie la règle de mise à jour en $\mathbf{w} := \mathbf{w} + \gamma \cdot y_k \cdot \mathbf{x}_k$

ENTRAÎNEMENT (TRAINING)

Algorithm 2: Training algorithm of the perceptron (rewriting)

Data: dataset = $\{(\mathbf{x}_i, y_i) : i = 1, \dots, K\}$

$\mathbf{w} := (0, \dots, 0, 0) = \mathbf{0}$

for $e = 1$ **to** nb_epochs **do**

for $k = 1$ **to** K **do**

if $y_k \cdot \sigma(\mathbf{w}^T \mathbf{x}_k) < 0$ **then**

$\mathbf{w} := \mathbf{w} + y_k \cdot \mathbf{x}_k$

end

end

end

return $\mathbf{w}$

- ▶ On peut utiliser un *learning rate* $\gamma > 0$, ce qui modifie la règle de mise à jour en $\mathbf{w} := \mathbf{w} + \gamma \cdot y_k \cdot \mathbf{x}_k$

ENTRAÎNEMENT (TRAINING)

Algorithm 2: Training algorithm of the perceptron (rewriting)

Data: dataset = $\{(\mathbf{x}_i, y_i) : i = 1, \dots, K\}$

$\mathbf{w} := (0, \dots, 0, 0) = \mathbf{0}$

for $e = 1$ *to* nb_epochs **do**

for $k = 1$ *to* K **do**

if $y_k \cdot \sigma(\mathbf{w}^T \mathbf{x}_k) < 0$ **then**

$\mathbf{w} := \mathbf{w} + y_k \cdot \mathbf{x}_k$

end

end

end

return $\mathbf{w}$

- ▶ On peut utiliser un *learning rate* $\gamma > 0$, ce qui modifie la règle de mise à jour en $\mathbf{w} := \mathbf{w} + \gamma \cdot y_k \cdot \mathbf{x}_k$

ENTRAÎNEMENT (TRAINING)

Algorithm 2: Training algorithm of the perceptron (rewriting)

Data: dataset = $\{(\mathbf{x}_i, y_i) : i = 1, \dots, K\}$

$\mathbf{w} := (0, \dots, 0, 0) = \mathbf{0}$

for $e = 1$ *to* nb_epochs **do**

for $k = 1$ *to* K **do**

if $y_k \cdot \sigma(\mathbf{w}^T \mathbf{x}_k) < 0$ **then**

$\mathbf{w} := \mathbf{w} + y_k \cdot \mathbf{x}_k$

end

end

end

return $\mathbf{w}$

- ▶ On peut utiliser un *learning rate* $\gamma > 0$, ce qui modifie la règle de mise à jour en $\mathbf{w} := \mathbf{w} + \gamma \cdot y_k \cdot \mathbf{x}_k$

ENTRAÎNEMENT (TRAINING)

Algorithm 2: Training algorithm of the perceptron (rewriting)

Data: dataset = $\{(\mathbf{x}_i, y_i) : i = 1, \dots, K\}$

$\mathbf{w} := (0, \dots, 0, 0) = \mathbf{0}$

for $e = 1$ *to* nb_epochs **do**

for $k = 1$ *to* K **do**

if $y_k \cdot \sigma(\mathbf{w}^T \mathbf{x}_k) < 0$ **then**

$\mathbf{w} := \mathbf{w} + y_k \cdot \mathbf{x}_k$

end

end

end

return $\mathbf{w}$

- On peut utiliser un *learning rate* $\gamma > 0$, ce qui modifie la règle de mise à jour en $\mathbf{w} := \mathbf{w} + \gamma \cdot y_k \cdot \mathbf{x}_k$

ENTRAÎNEMENT (TRAINING)

Algorithm 2: Training algorithm of the perceptron (rewriting)

Data: dataset = $\{(\mathbf{x}_i, y_i) : i = 1, \dots, K\}$

$\mathbf{w} := (0, \dots, 0, 0) = \mathbf{0}$

for $e = 1$ **to** nb_epochs **do**

for $k = 1$ **to** K **do**

if $y_k \cdot \sigma(\mathbf{w}^T \mathbf{x}_k) < 0$ **then**

$\mathbf{w} := \mathbf{w} + y_k \cdot \mathbf{x}_k$

end

end

end

return $\mathbf{w}$

- ▶ On peut utiliser un *learning rate* $\gamma > 0$, ce qui modifie la règle de mise à jour en $\mathbf{w} := \mathbf{w} + \gamma \cdot y_k \cdot \mathbf{x}_k$

TRAINING ALGORITHM

```
def train_perceptron(x, y, nb_epochs):  
  
    # modified inputs (extend x with a column of 1's)  
    x_tmp = torch.ones((x.shape[0], x.shape[1] + 1))  
    x_tmp[:, 1:] = x  
  
    # initial weights  
    w = torch.zeros(x.size(1) + 1)  
  
    # iterate over epochs  
    for e in range(nb_epochs):  
  
        nb_changes = 0  
  
        # iterate over train set  
        for i in range(x.shape[0]):  
  
            if w.dot(x_tmp[i]) * y[i] <= 0:  
  
                w = w + (y[i] * x_tmp[i, :])  
                nb_changes = nb_changes + 1  
  
        if nb_changes == 0:  
  
            break  
  
    return w
```


ILLUSTRATION GÉOMÉTRIQUE

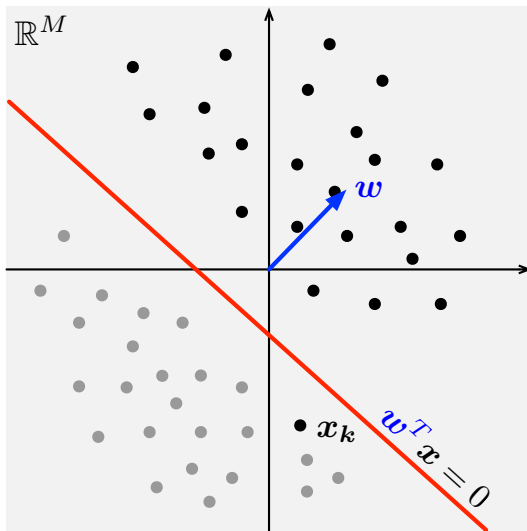


ILLUSTRATION GÉOMÉTRIQUE

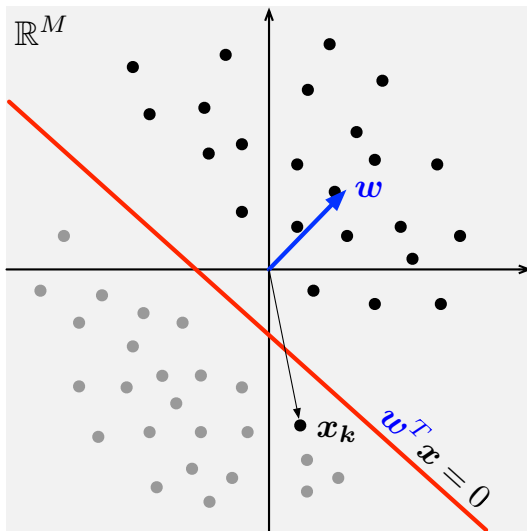


ILLUSTRATION GÉOMÉTRIQUE

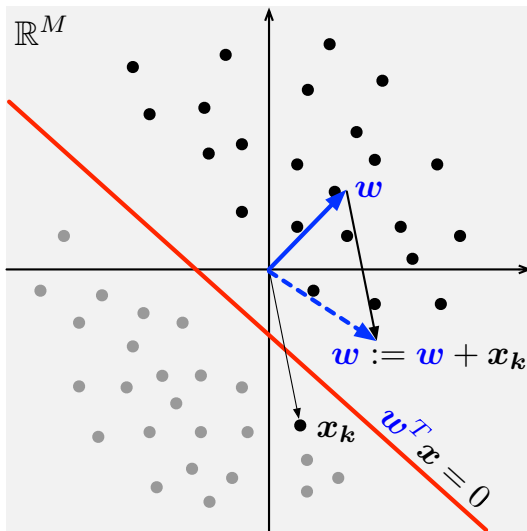


ILLUSTRATION GÉOMÉTRIQUE

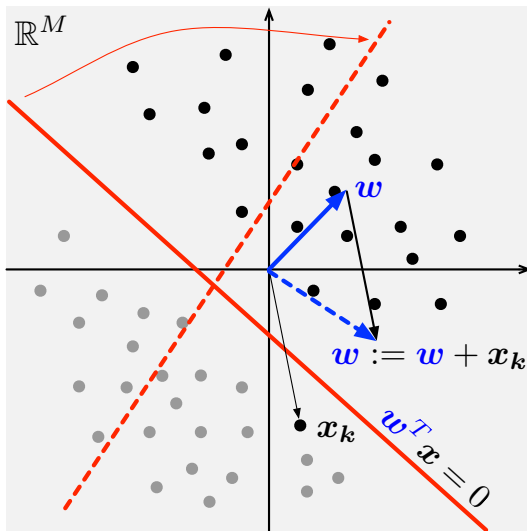


ILLUSTRATION GÉOMÉTRIQUE

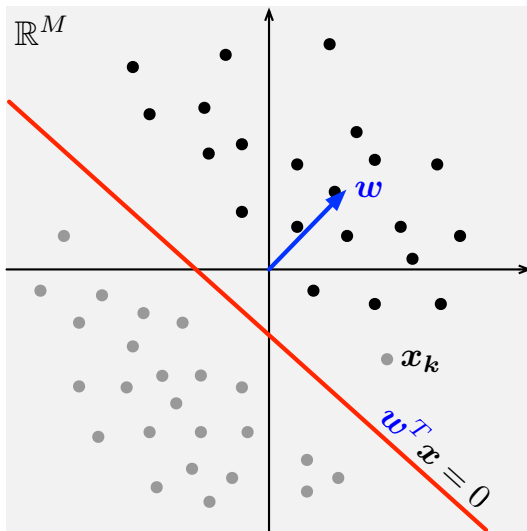


ILLUSTRATION GÉOMÉTRIQUE

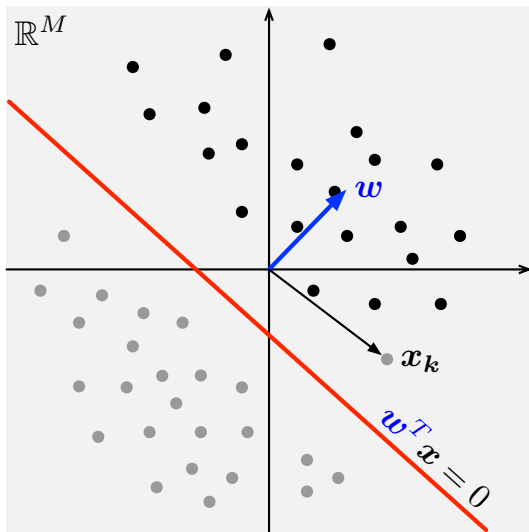


ILLUSTRATION GÉOMÉTRIQUE

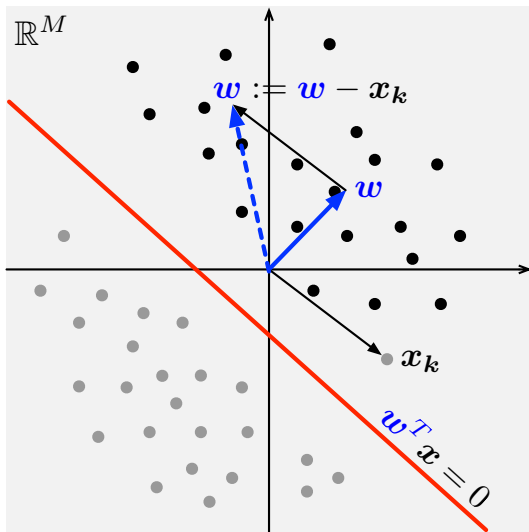
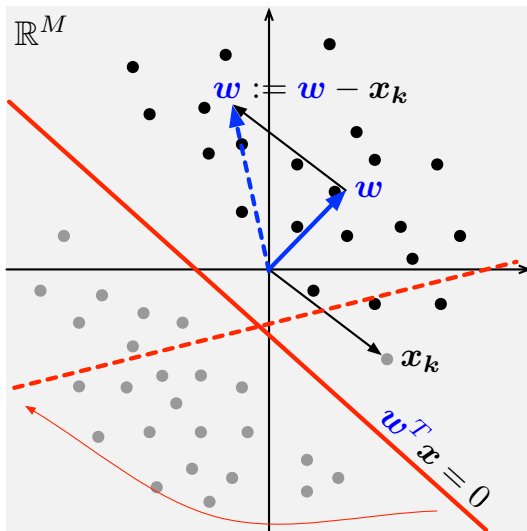
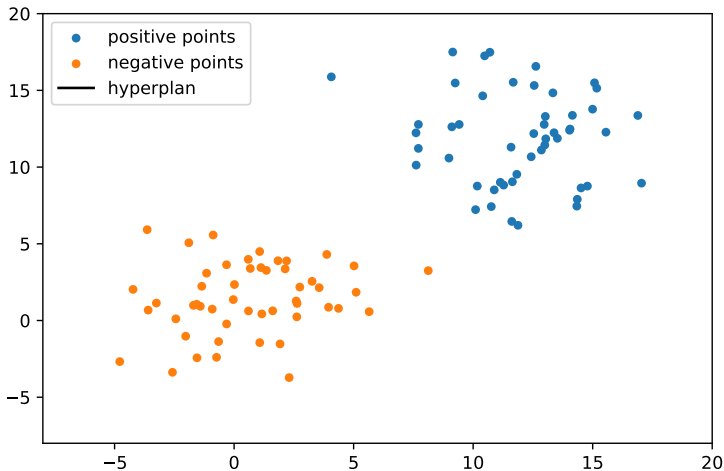


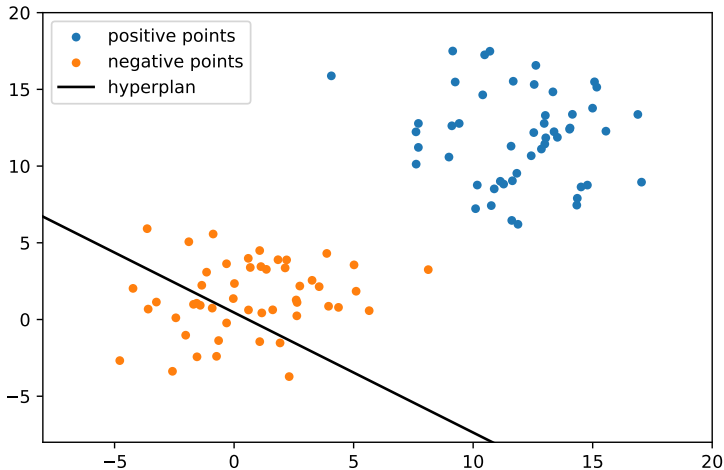
ILLUSTRATION GÉOMÉTRIQUE



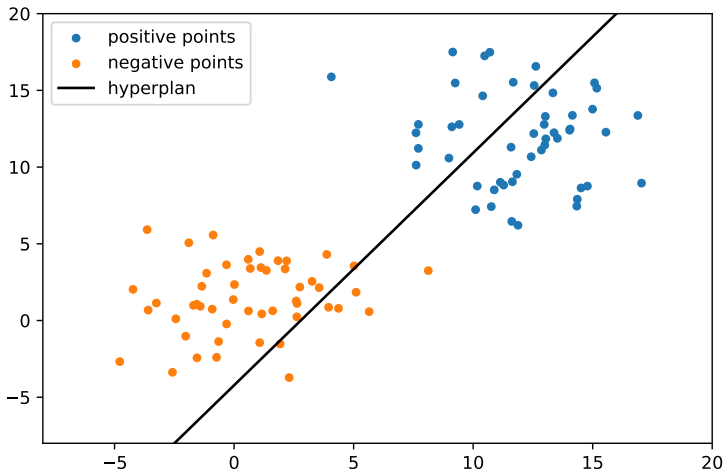
ENTRAÎNEMENT (TRAINING)



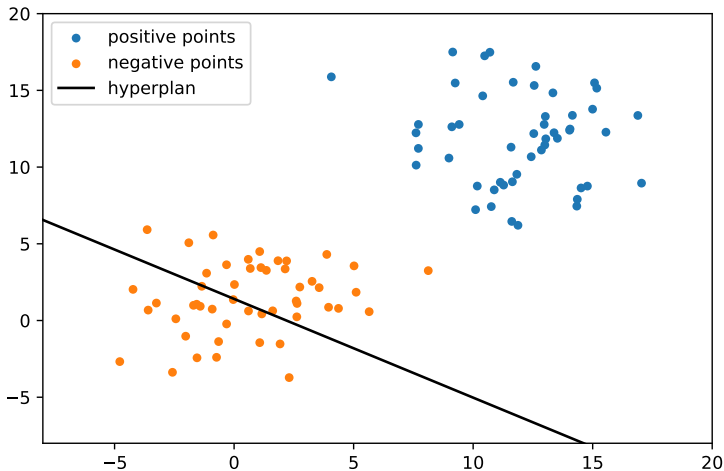
ENTRAÎNEMENT (TRAINING)



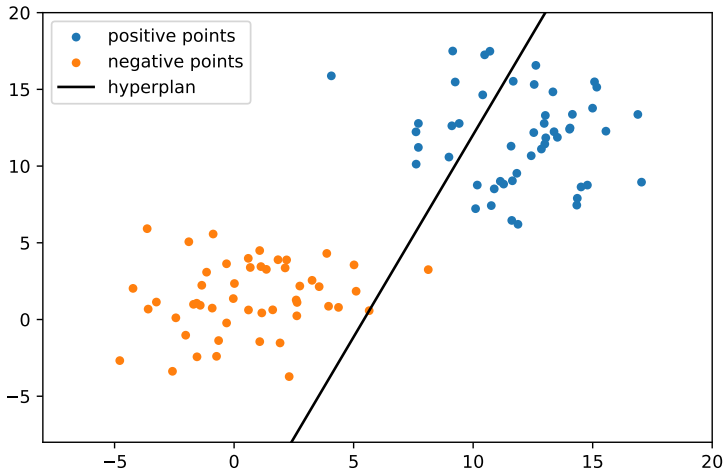
ENTRAÎNEMENT (TRAINING)



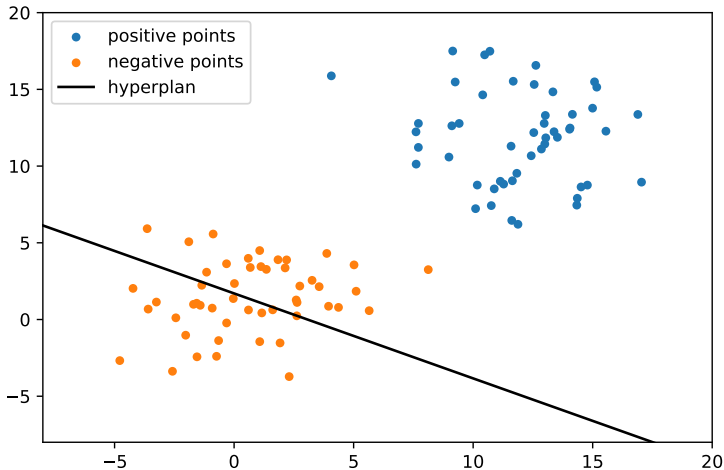
ENTRAÎNEMENT (TRAINING)



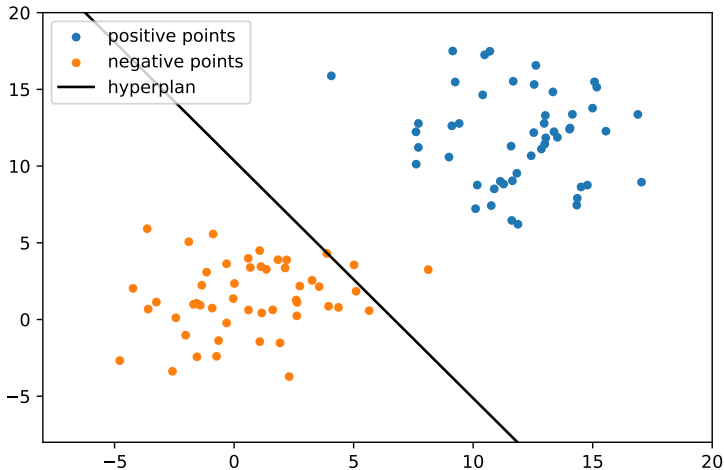
ENTRAÎNEMENT (TRAINING)



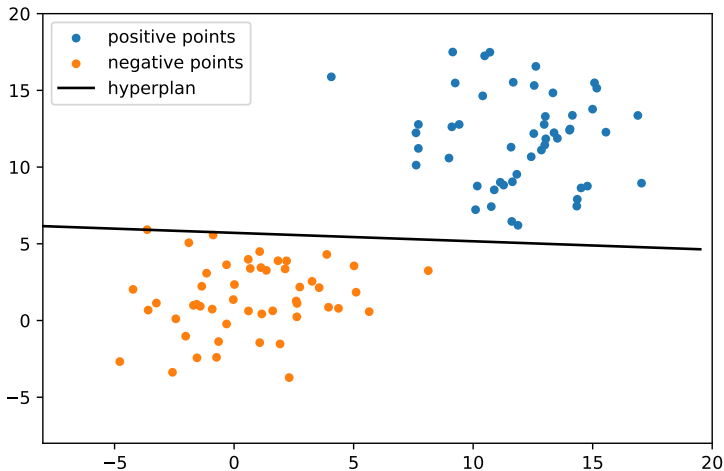
ENTRAÎNEMENT (TRAINING)



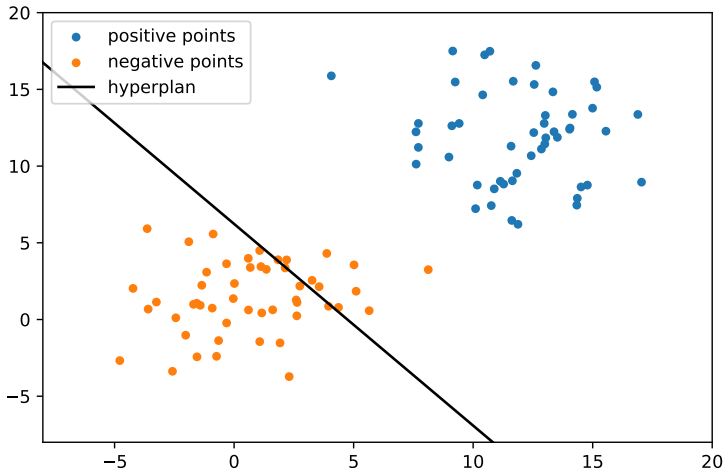
ENTRAÎNEMENT (TRAINING)



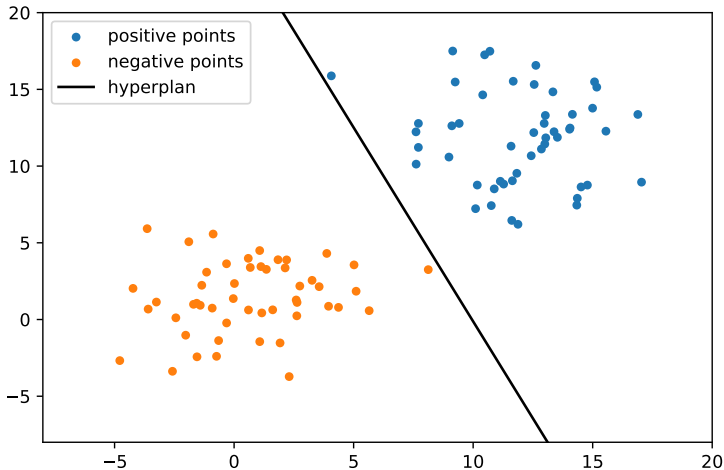
ENTRAÎNEMENT (TRAINING)



ENTRAÎNEMENT (TRAINING)

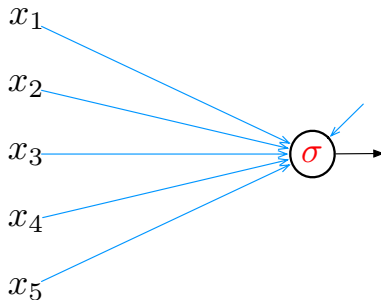


ENTRAÎNEMENT (TRAINING)



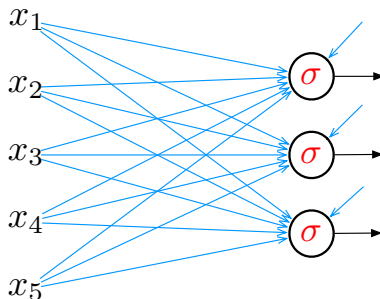
PERCEPTRON: 1 NEURONE

$$a = \sigma(\mathbf{w}^T \mathbf{x} + b) \text{ où } \mathbf{w} = (w_1, \dots, w_M)$$



PERCEPTRON: 1 COUCHE

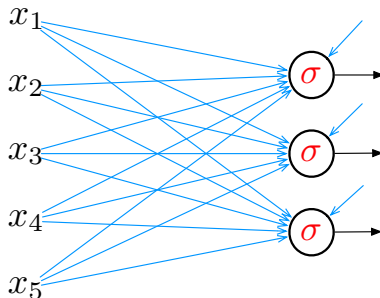
$$a_i = \sigma_i(\mathbf{w}_i^T \mathbf{x} + b_i) \text{ où } \mathbf{w}_i = (w_{i1}, \dots, w_{iM}), i = 1, 2, 3.$$



PERCEPTRON: 1 COUCHE

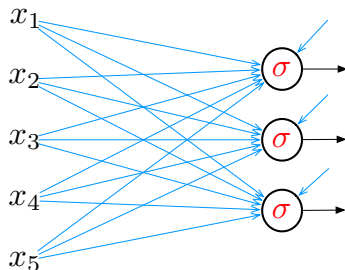
$$\mathbf{a} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}) \quad \text{où} \quad \mathbf{W} = \begin{pmatrix} \dots \mathbf{w}_1^T \dots \\ \dots \mathbf{w}_2^T \dots \\ \dots \mathbf{w}_3^T \dots \end{pmatrix} = \begin{pmatrix} w_{11} & \dots & w_{1M} \\ w_{21} & \dots & w_{2M} \\ w_{31} & \dots & w_{3M} \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix}$$

et $\sigma = (\sigma_1, \sigma_2, \sigma_3)$ appliquée composante par composante.



PERCEPTRON: 1 COUCHE

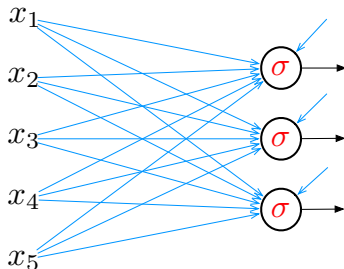
$$\begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix} = \sigma \left[\begin{pmatrix} w_{11} & w_{12} & w_{13} & w_{14} & w_{15} \\ w_{21} & w_{22} & w_{23} & w_{24} & w_{25} \\ w_{31} & w_{32} & w_{33} & w_{34} & w_{35} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{pmatrix} + \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix} \right]$$



► w_{ij} : poids de l'input j vers neurone i (et non de i vers j).

PERCEPTRON: 1 COUCHE

$$\begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix} = \sigma \left[\begin{pmatrix} w_{11} & w_{12} & w_{13} & w_{14} & w_{15} \\ w_{21} & w_{22} & w_{23} & w_{24} & w_{25} \\ w_{31} & w_{32} & w_{33} & w_{34} & w_{35} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{pmatrix} + \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix} \right]$$



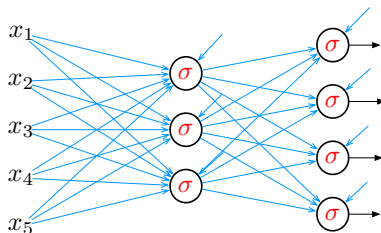
- w_{ij} : poids de l'input j vers neurone i (et non de i vers j).

PERCEPTRON: 2 COUCHES

$$a^{[1]} = \sigma \left(W^{[1]}x + b^{[1]} \right) := \sigma \left(z^{[1]} \right)$$

$$a^{[2]} = \sigma \left(W^{[2]}a^{[1]} + b^{[2]} \right) := \sigma \left(z^{[2]} \right)$$

où $W^{[i]} = \begin{pmatrix} \dots w_1^{[i]T} \dots \\ \dots \dots \dots \\ \dots w_{l_i}^{[i]T} \dots \end{pmatrix} = \begin{pmatrix} w_{11}^{[i]} & \dots & w_{1l_{i-1}}^{[i]} \\ \dots & \dots & \dots \\ w_{l_i1}^{[i]} & \dots & w_{l_i l_{i-1}}^{[i]} \end{pmatrix}$ et $b = \begin{pmatrix} b_1^{[i]} \\ \vdots \\ b_{l_i}^{[i]} \end{pmatrix}$ $i = 1, 2$

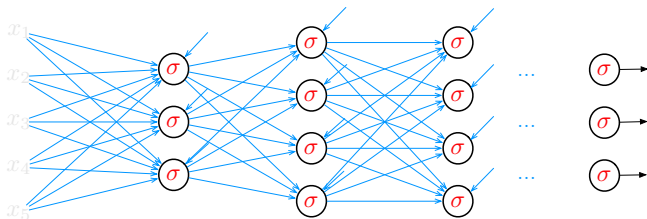


MULTI LAYER PERCEPTRON (MLP)

$$\begin{cases} \mathbf{a}^{[0]} = \mathbf{x} \\ \mathbf{z}^{[l]} = \mathbf{W}^{[l]} \mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}, \\ \mathbf{a}^{[l]} = \boldsymbol{\sigma}(\mathbf{z}^{[l]}) \end{cases} \quad l = 1, \dots, L$$

➤ $\mathbf{x}$ est l'input et $\mathbf{a}^{[L]}$ est l'output

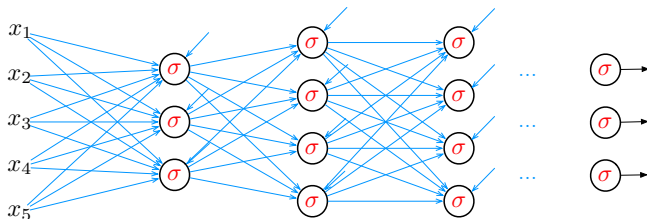
➤ $\mathbf{W}^{[l]} \in \mathbb{R}^{\lambda_l \times \lambda_{l-1}}$ et $\mathbf{b}^{[l]} \in \mathbb{R}^{\lambda_l}$, où λ_l taille de la couche l



MULTI LAYER PERCEPTRON (MLP)

$$\begin{cases} \mathbf{a}^{[0]} = \mathbf{x} \\ \mathbf{z}^{[l]} = \mathbf{W}^{[l]} \mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}, \\ \mathbf{a}^{[l]} = \sigma(\mathbf{z}^{[l]}) \end{cases} \quad l = 1, \dots, L$$

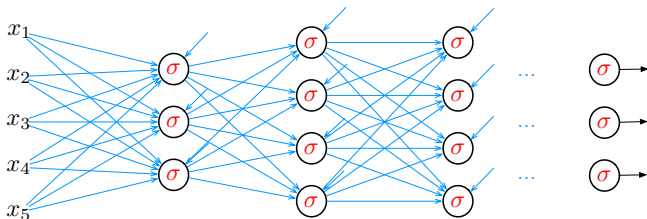
- ▶ $\mathbf{x}$ est l'input et $\mathbf{a}^{[L]}$ est l'output
- ▶ $\mathbf{W}^{[l]} \in \mathbb{R}^{\lambda_l \times \lambda_{l-1}}$ et $\mathbf{b}^{[l]} \in \mathbb{R}^{\lambda_l}$, où λ_l taille de la couche l
- ▶ $w_{ij}^{[l]}$: poids du neurone j (couche $l-1$) vers neurone i (couche l)



MULTI LAYER PERCEPTRON (MLP)

$$\begin{cases} \mathbf{a}^{[0]} = \mathbf{x} \\ \mathbf{z}^{[l]} = \mathbf{W}^{[l]} \mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}, \\ \mathbf{a}^{[l]} = \sigma(\mathbf{z}^{[l]}) \end{cases} \quad l = 1, \dots, L$$

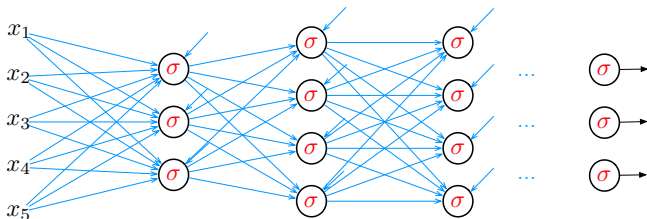
- ▶ $\mathbf{x}$ est l'input et $\mathbf{a}^{[L]}$ est l'output
- ▶ $\mathbf{W}^{[l]} \in \mathbb{R}^{\lambda_l \times \lambda_{l-1}}$ et $\mathbf{b}^{[l]} \in \mathbb{R}^{\lambda_l}$, où λ_l taille de la couche l
- ▶ $w_{ij}^{[l]}$: poids du neurone j (couche $l-1$) vers neurone i (couche l)



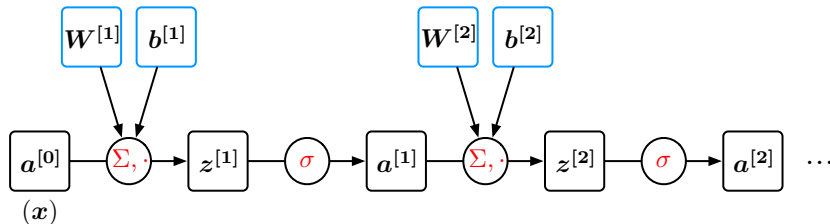
MULTI LAYER PERCEPTRON (MLP)

$$\begin{cases} \mathbf{a}^{[0]} = \mathbf{x} \\ \mathbf{z}^{[l]} = \mathbf{W}^{[l]} \mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}, \\ \mathbf{a}^{[l]} = \sigma(\mathbf{z}^{[l]}) \end{cases} \quad l = 1, \dots, L$$

- ▶ $\mathbf{x}$ est l'input et $\mathbf{a}^{[L]}$ est l'output
- ▶ $\mathbf{W}^{[l]} \in \mathbb{R}^{\lambda_l \times \lambda_{l-1}}$ et $\mathbf{b}^{[l]} \in \mathbb{R}^{\lambda_l}$, où λ_l taille de la couche l
- ▶ $w_{ij}^{[l]}$: poids du neurone j (couche $l-1$) vers neurone i (couche l)

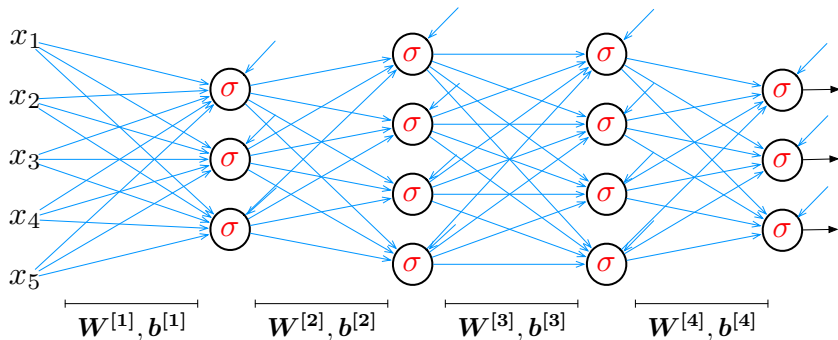


MLP: REPRÉSENTATION GRAPHIQUE



MLP: FORWARD PASS

$$\begin{cases} \mathbf{a}^{[0]} = \mathbf{x} \\ \mathbf{z}^{[l]} = \mathbf{W}^{[l]} \mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}, \\ \mathbf{a}^{[l]} = \sigma(\mathbf{z}^{[l]}) \end{cases} \quad l = 1, \dots, L$$



MLP: FORWARD PASS

Algorithm 3: MLP: forward pass

Data: dataset = $\{(x_i, y_i) : i = 1, \dots, N\}$

Inputs: MLP = $\{(W^{[l]}, b^{[l]}) : l = 1, \dots, L\}$

```
predictions = []
for i = 1 to N do
     $a^{[0]} = x_i$ 
    for l = 1 to L do                                     // forward pass
         $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$ 
         $a^{[l]} = \sigma(z^{[l]})$ 
    end
    predictions.append( $a^{[L]}$ )
end
return predictions
```

MLP: FORWARD PASS

Algorithm 3: MLP: forward pass

Data: dataset = $\{(x_i, y_i) : i = 1, \dots, N\}$

Inputs: MLP = $\{(W^{[l]}, b^{[l]}) : l = 1, \dots, L\}$

predictions = []

for $i = 1$ to N do

$a^{[0]} = x_i$

 for $l = 1$ to L do

$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$

$a^{[l]} = \sigma(z^{[l]})$

 end

 predictions.append($a^{[L]}$)

end

return predictions

// forward pass

MLP: FORWARD PASS

Algorithm 3: MLP: forward pass

Data: dataset = $\{(x_i, y_i) : i = 1, \dots, N\}$

Inputs: MLP = $\{(W^{[l]}, b^{[l]}) : l = 1, \dots, L\}$

predictions = []

for $i = 1$ **to** N **do**

$a^{[0]} = x_i$

for $l = 1$ **to** L **do**

$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$

$a^{[l]} = \sigma(z^{[l]})$

end

 predictions.append($a^{[L]}$)

end

return predictions

// forward pass

MLP: FORWARD PASS

Algorithm 3: MLP: forward pass

Data: dataset = $\{(x_i, y_i) : i = 1, \dots, N\}$

Inputs: MLP = $\{(W^{[l]}, b^{[l]}) : l = 1, \dots, L\}$

predictions = []

for $i = 1$ **to** N **do**

$a^{[0]} = x_i$

for $l = 1$ **to** L **do**

$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$

$a^{[l]} = \sigma(z^{[l]})$

end

 predictions.append($a^{[L]}$)

end

return predictions

// forward pass

MLP: FORWARD PASS

Algorithm 3: MLP: forward pass

Data: dataset = $\{(x_i, y_i) : i = 1, \dots, N\}$

Inputs: MLP = $\{(W^{[l]}, b^{[l]}) : l = 1, \dots, L\}$

predictions = []

for $i = 1$ **to** N **do**

$a^{[0]} = x_i$

for $l = 1$ **to** L **do**

$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$

$a^{[l]} = \sigma(z^{[l]})$

end

 predictions.append($a^{[L]}$)

end

return predictions

// forward pass

MLP: FORWARD PASS

Algorithm 3: MLP: forward pass

Data: dataset = $\{(x_i, y_i) : i = 1, \dots, N\}$ **Inputs:** MLP = $\{(W^{[l]}, b^{[l]}) : l = 1, \dots, L\}$

predictions = []

for $i = 1$ **to** N **do** $a^{[0]} = x_i$ **for** $l = 1$ **to** L **do** $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$ $a^{[l]} = \sigma(z^{[l]})$ **end** predictions.append($a^{[L]}$)**end****return** predictions

// forward pass

MLP: FORWARD PASS (BATCHED)

- Soit le dataset $S = \{(x_i, y_i) : i = 1, \dots, N\}$
- On peut *paralléliser* la forward pass en passant les data "batch par batch" (batches de taille $B = 32, 64, \dots$).
- Le i -ème batch $B_i = (X_i, Y_i)$ est composé de B inputs et outputs x_k et y_k alignés en deux matrices:

$$X_i = \begin{pmatrix} \vdots & \vdots & \cdots & \vdots \\ x_1 & x_2 & \cdots & x_B \\ \vdots & \vdots & \cdots & \vdots \end{pmatrix} \text{ et } Y_i = \begin{pmatrix} \vdots & \vdots & \cdots & \vdots \\ y_1 & y_2 & \cdots & y_B \\ \vdots & \vdots & \cdots & \vdots \end{pmatrix}$$

MLP: FORWARD PASS (BATCHED)

- ▶ Soit le dataset $S = \{(x_i, y_i) : i = 1, \dots, N\}$
- ▶ On peut *paralléliser* la forward pass en passant les data “batch par batch” (batches de taille $B = 32, 64, \dots$).
- ▶ Le i -ème batch $B_i = (X_i, Y_i)$ est composé de B inputs et outputs x_k et y_k alignés en deux matrices:

$$X_i = \begin{pmatrix} \vdots & \vdots & \cdots & \vdots \\ x_1 & x_2 & \cdots & x_B \\ \vdots & \vdots & \cdots & \vdots \end{pmatrix} \text{ et } Y_i = \begin{pmatrix} \vdots & \vdots & \cdots & \vdots \\ y_1 & y_2 & \cdots & y_B \\ \vdots & \vdots & \cdots & \vdots \end{pmatrix}$$

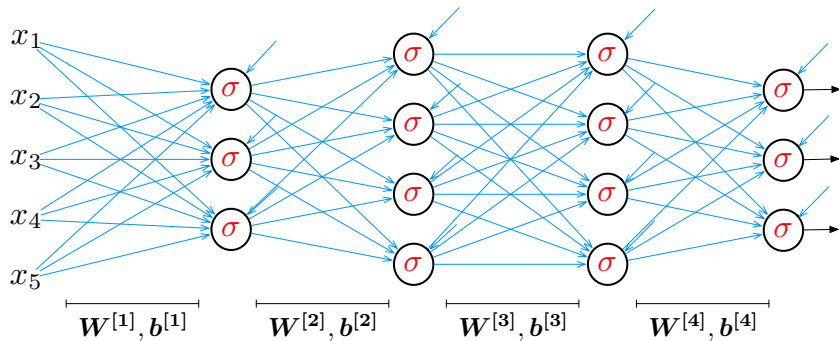
MLP: FORWARD PASS (BATCHED)

- ▶ Soit le dataset $S = \{(x_i, y_i) : i = 1, \dots, N\}$
- ▶ On peut *paralléliser* la forward pass en passant les data "batch par batch" (batches de taille $B = 32, 64, \dots$).
- ▶ Le i -ème batch $B_i = (X_i, Y_i)$ est composé de B inputs et outputs x_k et y_k alignés en deux matrices:

$$X_i = \begin{pmatrix} \vdots & \vdots & \dots & \vdots \\ x_1 & x_2 & \dots & x_B \\ \vdots & \vdots & \dots & \vdots \end{pmatrix} \text{ et } Y_i = \begin{pmatrix} \vdots & \vdots & \dots & \vdots \\ y_1 & y_2 & \dots & y_B \\ \vdots & \vdots & \dots & \vdots \end{pmatrix}$$

MLP: FORWARD PASS (BATCHED)

$$\begin{cases} \mathbf{A}^{[0]} = \mathbf{X}_i \\ \mathbf{Z}^{[l]} = \mathbf{W}^{[l]} \mathbf{A}^{[l-1]} \oplus \mathbf{b}^{[l]}, \\ \mathbf{A}^{[l]} = \sigma(\mathbf{Z}^{[l]}) \end{cases} \quad l = 1, \dots, L$$



MLP: FORWARD PASS (BATCHED)

Algorithm 4: MLP: forward pass (batched)

Data: $\text{dataloader} = \{B_i = (X_i, Y_i) : i = 1, \dots, B\}$

Inputs: $\text{MLP} = \{(W^{[l]}, b^{[l]}) : l = 1, \dots, L\}$

```
predictions = []
for i = 1 to B do                                // loop over batches
     $A^{[0]} = X_i$ 
    for l = 1 to L do                            // forward pass
         $Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]}$ 
         $A^{[l]} = \sigma(Z^{[l]})$ 
    end
    predictions.append( $A^{[L]}$ )
end
return concat(predictions)
```

MLP: FORWARD PASS (BATCHED)

Algorithm 4: MLP: forward pass (batched)

Data: dataloader = $\{B_i = (X_i, Y_i) : i = 1, \dots, B\}$

Inputs: MLP = $\{(W^{[l]}, b^{[l]}) : l = 1, \dots, L\}$

predictions = []

```
for i = 1 to B do                                // loop over batches
     $A^{[0]} = X_i$ 
    for l = 1 to L do                             // forward pass
         $Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]}$ 
         $A^{[l]} = \sigma(Z^{[l]})$ 
    end
    predictions.append( $A^{[L]}$ )
end
return concat(predictions)
```

MLP: FORWARD PASS (BATCHED)

Algorithm 4: MLP: forward pass (batched)

Data: dataloader = $\{B_i = (X_i, Y_i) : i = 1, \dots, B\}$

Inputs: MLP = $\{(W^{[l]}, b^{[l]}) : l = 1, \dots, L\}$

predictions = []

for $i = 1$ **to** B **do**

// loop over batches

$A^{[0]} = X_i$

for $l = 1$ **to** L **do**

// forward pass

$Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]}$

$A^{[l]} = \sigma(Z^{[l]})$

end

 predictions.append($A^{[L]}$)

end

return concat(predictions)

MLP: FORWARD PASS (BATCHED)

Algorithm 4: MLP: forward pass (batched)

Data: dataloader = $\{B_i = (X_i, Y_i) : i = 1, \dots, B\}$

Inputs: MLP = $\{(W^{[l]}, b^{[l]}) : l = 1, \dots, L\}$

predictions = []

for $i = 1$ **to** B **do**

// loop over batches

$A^{[0]} = X_i$

for $l = 1$ **to** L **do**

// forward pass

$Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]}$

$A^{[l]} = \sigma(Z^{[l]})$

end

 predictions.append($A^{[L]}$)

end

return concat(predictions)

MLP: FORWARD PASS (BATCHED)

Algorithm 4: MLP: forward pass (batched)

Data: dataloader = $\{B_i = (X_i, Y_i) : i = 1, \dots, B\}$

Inputs: MLP = $\{(W^{[l]}, b^{[l]}) : l = 1, \dots, L\}$

predictions = []

for $i = 1$ **to** B **do**

// loop over batches

$A^{[0]} = X_i$

for $l = 1$ **to** L **do**

// forward pass

$Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]}$

$A^{[l]} = \sigma(Z^{[l]})$

end

 predictions.append($A^{[L]}$)

end

return concat(predictions)

MLP: FORWARD PASS (BATCHED)

Algorithm 4: MLP: forward pass (batched)

Data: dataloader = $\{B_i = (X_i, Y_i) : i = 1, \dots, B\}$ **Inputs:** MLP = $\{(W^{[l]}, b^{[l]}) : l = 1, \dots, L\}$

predictions = []

for $i = 1$ **to** B **do**

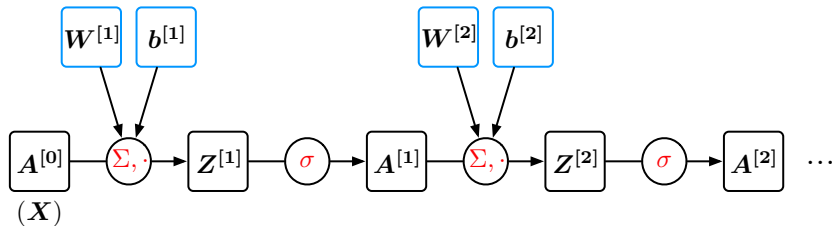
// loop over batches

 $A^{[0]} = X_i$ **for** $l = 1$ **to** L **do**

// forward pass

 $Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]}$ $A^{[l]} = \sigma(Z^{[l]})$ **end** predictions.append($A^{[L]}$)**end****return** concat(predictions)

MLP: REPRÉSENTATION GRAPHIQUE (BATCHED)



carrés = variables

ronds = opérations

BIBLIOGRAPHIE



Fleuret, F. (2022).
Deep Learning Course.



Rosenblatt, F. (1957).
The perceptron: A perceiving and recognizing automaton.
Technical Report 85-460-1, Cornell Aeronautical Laboratory, Ithaca, New York.



Rosenblatt, F. (1958).
The perceptron: A probabilistic model for information storage and organization in the brain.
Psychological Review, 65(6):386–408.